# Noah Passmore Game Project

## Contents:

Noah Passmore Game Project ..... 1
Contents: ..... 1
Introduction: ..... 1
Overview: ..... 2
Controls: ..... 2
Game Rules: ..... 2
Structure: ..... 3
Game Flow and Logic: ..... 3
Graphics: ..... 4
Challenges: ..... 4
Conclusion: ..... 4

## Introduction:

For this project, I developed a simple Flappy Bird-style game using the raylib graphics library, the C language. The aim of the project was to create a playable game while demonstrating my understanding of core programming concepts such as loops functions, and structure

The game is based on the original Flappy Bird idea where the player controls a bird that must pass through gaps between pipes without hitting them. As the game progresses, the speed increases, making it more challenging.

## Overview:

The game is designed as a basic 2D arcade game. The player controls a bird that constantly falls due to gravity. The objective is to keep the bird in the air and guide it through gaps between pipes.

The pipes move from right to left across the screen, and the player scores points for each pipe they successfully pass. The game ends if the bird collides with a pipe, the ground, or the top of the screen.

## Controls:

Spacebar - Jump

R – Restart

## Game Rules:

Avoid hitting pipes

Stay within screen bounds

Score increases when passing pipes

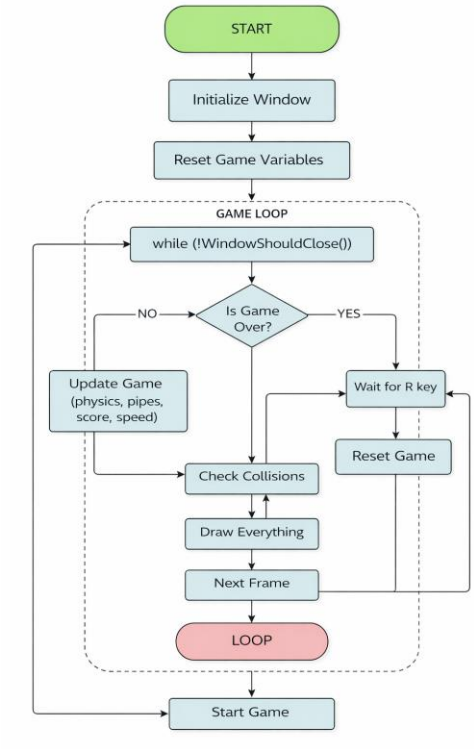
Game speed increases over time

Game ends on collision

## Structure:

The program includes a main game loop, bird and pipe structures, a reset function, update logic, and drawing logic.

## Game Flow and Logic:




The game starts by setting up the window and resetting variables. It then enters a loop that updates movement, checks collisions, updates score and redraws the screen continuously.

## Graphics:

The game uses simple shapes:

Bird: circle

Pipes: rectangles

Ground: rectangle

Text for score and messages

## Challenges:

During the development of this project, I came across a few challenges. One of the main difficulties was setting up raylib correctly and ensuring that the compiler and library paths were configured properly as small mistakes would prevent the program from building. I also had issues with file paths when loading assets, which made debugging more time-consuming than expected. Implementing collision detection was another challenge, as it required careful handling of hitboxes to ensure the game behaved correctly without unfair collisions. Getting the movement and physics to feel smooth and responsive took some trial and error, particularly when adjusting gravity and jump strength. Overall, debugging both graphical and logical issues was a key part of the development process and helped improve my understanding of how the game works.

## Conclusion:

This project helped me understand how to build a simple interactive game using C. It demonstrates key programming concepts and results in a playable game with increasing difficulty.